



ΗΥ340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

**ΠΑΝΕΠΙΣΤΗΜΙΟ ΚΡΗΤΗΣ,
ΣΧΟΛΗ ΘΕΤΙΚΩΝ ΕΠΙΣΤΗΜΩΝ,
ΤΜΗΜΑ ΕΠΙΣΤΗΜΗΣ ΥΠΟΛΟΓΙΣΤΩΝ**

```
VAR i:Integer;  
  
FUNCTION(Symbol) replicate  
  
    x = (function(x,y){return x+y;});  
  
    class DelFunctor: public std::unary_function<
```

ΔΙΔΑΣΚΩΝ

Αντώνιος Σαββίδης



HY340 : ΓΛΩΣΣΕΣ ΚΑΙ ΜΕΤΑΦΡΑΣΤΕΣ

Διάλεξη 4η ΣΥΝΤΑΚΤΙΚΗ ΑΝΑΛΥΣΗ - I



Περιεχόμενα

- *Γραμματικές ανεξάρτητες συμφραζομένων*
- Δέντρα συντακτικής ανάλυσης
- Εξάλειψη διφορούμενης ανάλυσης



Γραμματικές ανεξάρτητες συμφραζομένων (1/12)

- Τα στοιχεία των περισσότερων γλωσσών προγραμματισμού έχουν μία αναδρομική δομή η οποία μπορεί να οριστεί με γραμματικές ανεξάρτητες συμφραζομένων – context free grammars (CFG)
- Οι κανονικές εκφράσεις αδυνατούν να περιγράψουν «φωλιασμένο» συντακτικό
 - Δεν έχουν τρόπο καταγραφής και μνήμης του τι έχει ήδη αναγνωριστεί από τα σύμβολα εισόδου
 - π.χ., καμία κανονική έκφραση δεν μπορεί να περιγράψει τη γλώσσα $a^n b^n$, για οποιοδήποτε n , ενώ η εξής CFG μπορεί:
 - ◆ $S \rightarrow a S b \mid \varepsilon$
 - Για $a = \{\text{if και } b = \text{else}\}$, η παραπάνω γλώσσα αφορά το σύνολο των φωλιασμένων *if...else* statements $(\{\text{if}\}^n (\text{else})^n$
 - ◆ **{if {if {if else} else} else}**



Γραμματικές ανεξάρτητες συμφραζομένων (2/12)

- Ένα παράδειγμα κλασικού προγραμματιστικού στοιχείου είναι οι εντολές διακλάδωσης. Εάν S_1 και S_2 είναι τύπου *εντολή* και E είναι τύπου *έκφραση*, τότε το παρακάτω είναι επίσης τύπου *εντολή*:
 - **if E then S_1 else S_2**
- Όπως είδαμε, αυτές οι εντολές δε μπορούν να οριστούν μέσω κανονικών εκφράσεων - οι κανονικές εκφράσεις είναι κατάλληλες μόνο για την περιγραφή των (κανονικών) λεξικογραφικών στοιχείων.
- Χρησιμοποιώντας την συντακτική μεταβλητή *stmt* να αντιπροσωπεύει την κλάση των εντολών και την μεταβλητή *expr* να αντιπροσωπεύει την κλάση των εκφράσεων, ο προηγούμενος γραμματικός κανόνας γράφεται ως εξής:
 - ***stmt* $\rightarrow$ if *expr* then *stmt* else *stmt***



Γραμματικές ανεξάρτητες συμφραζομένων (3/12)

Μια γραμματική χωρίς συμφραζόμενα είναι μία τετράδα $G = (V, \Sigma, R, S)$ που ορίζονται ως εξής:

- V είναι ένα αλφάβητο από σύμβολα
- Σ είναι το σύνολο των τερματικών (*terminals*) συμβόλων, υποσύνολο του V (θεωρήστε το Σ ως το σύνολο των λεξικογραφικών στοιχείων)
- Το σύνολο $V - \Sigma$ περιέχει τα λεγόμενα μη τερματικά στοιχεία (*non-terminals*)
- R είναι το σύνολο των κανόνων παραγωγής της γραμματικής (*grammar production rules*), το οποίο είναι ένα πεπερασμένο υποσύνολο του $(V - \Sigma) \times V^*$
- S είναι το αρχικό σύμβολο (*start symbol*) και είναι στοιχείο του $V - \Sigma$
- Ένας κανόνας ή παραγωγή $(A, q) \in R$ γράφεται συμβολικά ως $A \rightarrow q$



Γραμματικές ανεξάρτητες συμφραζομένων (4/12)

- Τα τερματικά σύμβολα (terminals) είναι τα βασικά συστατικά στοιχεία από τα οποία αποτελούνται οι λέξεις
- Ο όρος “token” θα είναι συνώνυμος του “terminal” όταν αναφερόμαστε σε γραμματικές γλωσσών προγραμματισμού
- Έτσι, τα **if**, **then**, **else** είναι *tokens* στο χώρο της *λεξικογραφικής ανάλυσης* και *terminals* στο πεδίο της *συντακτικής ανάλυσης*



Γραμματικές ανεξάρτητες συμφραζομένων (5/12)

- Τα μη τερματικά σύμβολα (non-terminals) είναι ένα είδος συντακτικών μεταβλητών που αντιστοιχούν σε σύνολα από λέξεις (συνήθως απειροσύνολα)
 - π.χ. το μη τερματικό σύμβολο *stmt* αντιστοιχεί σε σύνολο που περιέχει λέξεις όπως (C, C++, Java, alpha):
 - ♦ `if (x>y) { x = y-z; y = z-x; }`
 - ♦ `x = y-z;`
 - ♦ `y = z-x;`
 - ♦ `{ x = y-z; y = z-x; }`
- Τα *μη τερματικά σύμβολα*, ορίζοντας σύνολα λέξεων, βοηθούν *στον προσδιορισμό της γλώσσας* που παράγεται από μία γραμματική
- Επιπλέον *επιβάλλουν* μία *ιεραρχική δομή στη γλώσσα* η οποία είναι πολύτιμη τόσο για τη συντακτική ανάλυση, όσο και για τη μετάφραση



Γραμματικές ανεξάρτητες συμφραζομένων (6/12)

- Οι γραμματικές παραγωγές ορίζουν τον τρόπο με τον οποίο παράγουμε με συνδυασμούς terminals και non-terminals λέξεις της γλώσσας

Η γραμματική με τις παρακάτω παραγωγές ορίζει απλές αριθμητικές εκφράσεις, με terminals **id + - * / ()**, non-terminals *expr* και *op*, ενώ *expr* είναι το start symbol.

$expr \rightarrow expr\ op\ expr$

$expr \rightarrow (expr)$

$expr \rightarrow -\ expr$

$expr \rightarrow id$

$op \rightarrow +$

$op \rightarrow -$

$op \rightarrow *$

$op \rightarrow /$

Μπορείς να παράγεις ένα expression στη γλώσσα παίρνοντας δύο expressions και ενώνοντάς τα με ένα *op*.

Ένα expression μπορεί να είναι ένα αναγνωριστικό όνομα.

Ένα *op* μπορεί να είναι οποιοδήποτε από τα + - * /



Γραμματικές ανεξάρτητες συμφραζομένων (7/12)

- Ξεκινώντας με την προηγούμενη γραμματική
 - θεωρώντας ότι το **id** αντιστοιχεί στον λεξικογραφικό κανόνα που ορίζεται από την κανονική έκφραση **letter (letter | digit)***
 - μπορούμε να παράγουμε τις ακόλουθες εκφράσεις:
 - ➔ φαίνεται ότι μία γραμματική συνιστά και ένα εργαλείο τεκμηρίωσης των ορθών εκφράσεων μίας γλώσσας προγραμματισμού
 - ➔ δικαιολογείται και ο όρος «γραμματική παραγωγή» που χρησιμοποιείται όπως ο «γραμματικός κανόνας»

Παραγόμενη λέξη	Εφαρμοζόμενη παραγωγή
myVar	$expr \rightarrow id$
x	$expr \rightarrow id$
-myVar	$-expr$
-myVar + x	$expr \rightarrow expr \text{ op } expr, \text{ op } \rightarrow +$
(-myVar + x) * x	$expr \rightarrow expr \text{ op } expr, \text{ op } \rightarrow *$
((-myVar + x) * x)	$expr \rightarrow (expr)$
-((-myVar + x) * x)	$expr \rightarrow -expr$
y	$expr \rightarrow id$
-((-myVar + x) * x) / y	$expr \rightarrow expr \text{ op } expr, \text{ op } \rightarrow /$



Γραμματικές ανεξάρτητες συμφραζομένων (8/12)

■ Παραγωγές (1/3)

- Κάθε γραμματική παραγωγή του R μπορεί να θεωρηθεί ως ένας κανόνας αντικατάστασης βάση του οποίου:
 - ◆ το **μη τερματικό** σύμβολο στα **αριστερά**
 - ◆ **αντικαθίσταται από** τη **λέξη** που υπάρχει **στα δεξιά**
- Με αυτούς τους κανόνες αντικατάστασης μπορούμε ξεκινώντας από το S , το αρχικό μη τερματικό σύμβολο
 - εφαρμόζοντας αναδρομικά την αντικατάσταση για κάθε μη τερματικό σύμβολο
 - επιλέγοντας όποιες παραγωγές εμείς επιθυμούμε
 - να παράγουμε πιο πολύπλοκες λέξεις
 - οι οποίες συνήθως είναι απειράριθμες



Γραμματικές ανεξάρτητες συμφραζομένων (9/12)

■ Παραγωγές (2/3)

Π.χ. για την προηγούμενη γραμματική, χρησιμοποιώντας το E στη θέση του $expr$ για συντομία έκφρασης, μπορούμε να έχουμε μία τέτοια απλή αντικατάσταση του E (που είναι και το start symbol) από το $-E$ μέσω του κανόνα $E \rightarrow -E$. Αυτή η συγκεκριμένη αντικατάσταση συμβολίζεται και ονομάζεται ως εξής:

$$\underline{E \Rightarrow -E, \text{ δηλ. «το } E \text{ παράγει το } -E\text{»}}$$

Ομοίως, η παραγωγή $E \rightarrow (E)$ υποδεικνύει τη δυνατότητα αντικατάστασης ενός E , σε οποιαδήποτε λέξη γραμματικών συμβόλων, από το (E) , έτσι μπορούμε να έχουμε:

$$\underline{E * E \Rightarrow (E) * E \text{ καθώς και } E * E \Rightarrow E * (E)}$$

Μπορούμε λαμβάνοντας ένα συγκεκριμένο E μέσα σε μία γραμματική λέξη να εφαρμόσουμε επαναληπτικά παραγωγές με οποιαδήποτε σειρά επιθυμούμε και να λάβουμε μία ακολουθία από αντικαταστάσεις. Π.χ.

$$\underline{E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E * E) \Rightarrow -(\text{id} * E) \Rightarrow -(\text{id} * \text{id})}$$

Ονομάζουμε μία τέτοια ακολουθία από αντικαταστάσεις βάσει γραμματικών κανόνων *μία παραγωγή* του $-(\text{id} * \text{id})$ από το E .



Γραμματικές ανεξάρτητες συμφραζομένων (10/12)

■ Παραγωγές (3/3)

- Γενικεύοντας τους ορισμούς λέμε ότι: $\alpha A \beta \Rightarrow \alpha \gamma \beta$ εάν το $A \rightarrow \gamma$ είναι παραγωγή και α, β είναι λέξεις γραμματικών συμβόλων.
- Εάν $\alpha_1 \Rightarrow \alpha_2 \Rightarrow \dots \Rightarrow \alpha_n$ τότε λέμε ότι το α_1 παράγει το α_n .
- Ο συμβολισμός $\Rightarrow$ σημαίνει «παράγει σε ένα βήμα», ενώ ομοίως $\Rightarrow^*$ σημαίνει «παράγει σε μηδέν ή περισσότερα βήματα» και $\Rightarrow^+$ «παράγει σε ένα ή περισσότερα βήματα». Έτσι ισχύουν:

$$\alpha \Rightarrow^* \alpha, \text{ για κάθε λέξη } \alpha$$
$$\text{Εάν } \alpha \Rightarrow^* \beta \text{ και } \beta \Rightarrow^* \gamma \text{ τότε } \alpha \Rightarrow^* \gamma$$

- Επιπλέον δίνουμε και τους εξής ορισμούς:
 - **Φράση – phrase**, κάθε λέξη που παράγεται από ένα μη τερματικό σύμβολο
 - **Προτασιακός τύπος – sentential form**, κάθε λέξη που παράγεται από το αρχικό σύμβολο της γραμματικής
 - **Πρόταση – sentence**, κάθε προτασιακός τύπος που περιέχει μόνο τερματικά σύμβολα
 - **Γλώσσα – language**, το σύνολο όλων των προτάσεων μίας γραμματικής (συνήθως άπειρο), π.χ. η C++ είναι το σύνολο όλων των συντακτικά νόμιμων C++ προγραμμάτων



Γραμματικές ανεξάρτητες συμφραζομένων (11/12)

■ Ειδικές κατηγορίες παραγωγών

Ισχυρισμός: το $\text{id}*(\text{id}+\text{id})$ είναι πρόταση της γραμματικής (που είδαμε πριν).
Επίλυση: βρες μία οποιαδήποτε παραγωγή $S \Rightarrow^* \text{id}*(\text{id}+\text{id})$
Μία λύση: $E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (E + E) \Rightarrow \text{id} * (E + E) \Rightarrow \text{id} * (\text{id} + E) \Rightarrow \text{id} * (\text{id} + \text{id})$
Άλλη λύση: $E \Rightarrow E * E \Rightarrow \text{id} * E \Rightarrow \text{id} * (E) \Rightarrow \text{id} * (E + E) \Rightarrow \text{id} * (\text{id} + E) \Rightarrow \text{id} * (\text{id} + \text{id})$

Η δεύτερη λύση έχει ένα ενδιαφέρον χαρακτηριστικό: μόνο το αριστερότερο μη τερματικό γραμματικό σύμβολο της κάθε φράσης αντικαθίσταται. Τέτοιου είδους παραγωγές ονομάζονται *αριστερές* – leftmost derivations, συμβολικά $\Rightarrow_{lm}$, π.χ. μπορεί να έχουμε $\Rightarrow_{lm}^*$.

Κάθε προτασιακός τύπος παραγόμενος από αριστερές παραγωγές λέγεται *αριστερός προτασιακός τύπος*. Παρόμοια ορίζονται και οι δεξιές παραγωγές – rightmost derivations και δεξιοί προτασιακοί τύποι.

Μπορεί για την ίδια πρόταση να έχουμε πολλαπλές εναλλακτικές παραγωγές, ακόμη και εάν τις περιορίσουμε σε αριστερές ή δεξιές παραγωγές.

$E \rightarrow \text{id}$
 $E \rightarrow \text{id} * \text{id}$
 $E \rightarrow E * E$

$E \rightarrow E * E \rightarrow \text{id} * E \rightarrow \text{id} * \text{id} * \text{id}$
 $E \rightarrow E * E \rightarrow \text{id} * \text{id} * E \rightarrow \text{id} * \text{id} * \text{id}$
 Δύο αριστερές παραγωγές για την ίδια πρόταση



Γραμματικές ανεξάρτητες συμφραζομένων (12/12)

- *Η αξία όσων είπαμε στη μετάφραση*
 - Φαίνεται πλέον ότι οι γραμματικές τεκμηριώνουν και οριοθετούν τις νόμιμες προτάσεις κώδικα για μία γλώσσα προγραμματισμού
 - ◆ Πολύτιμο για τον χρήστη της γλώσσας, αλλιώς μόνο «εμπειρικά» και «διαισθητικά» αντιλαμβάνεστε ότι γράφετε σωστό συντακτικό
 - Επιπλέον, δεδομένης μίας υποτιθέμενης πρότασης για μία συγκεκριμένη γλώσσα προγραμματισμού (π.χ. υλοποίησης μίας function σε ένα αρχείο, για τη γλώσσα C)
 - ◆ Ο μεταφραστής θα πρέπει να εντοπίσει μία παραγωγή =>* αυτής με έναρξη το αρχικό σύμβολο της γλώσσας (ας το πούμε *program*) – θα δούμε αργότερα με ποιο τρόπο.
 - ↓ *Εάν δε βρεθεί τέτοια παραγωγή τότε προφανώς το πρόγραμμα εισόδου έχει συντακτικά λάθη*

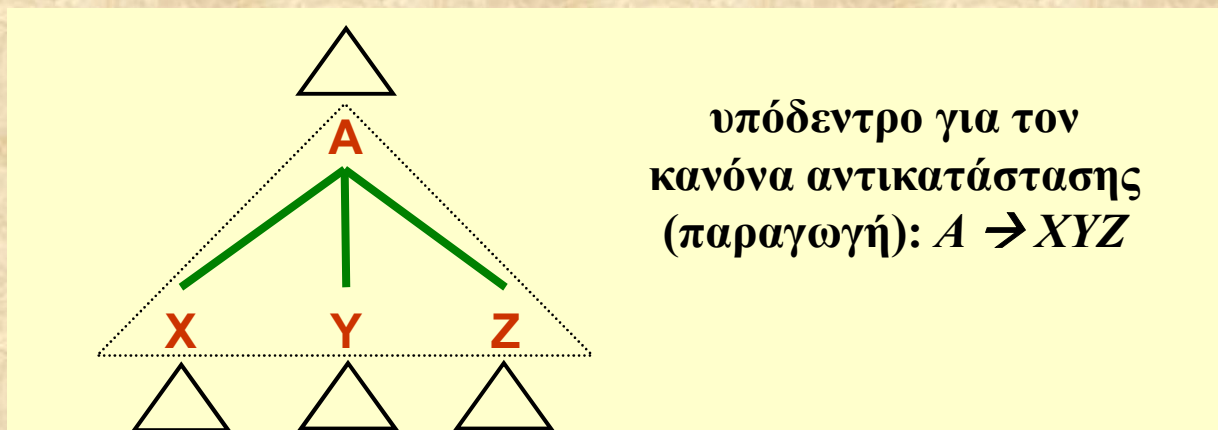


Περιεχόμενα

- Γραμματικές ανεξάρτητες συμφραζομένων
- *Δέντρα συντακτικής ανάλυσης*
- Εξάλειψη διφορούμενης ανάλυσης

Δέντρα συντακτικής ανάλυσης (1/4)

- Ένα δέντρο συντακτικής ανάλυσης – *parse tree*, δείχνει με γραφικό τρόπο πως το αρχικό σύμβολο της γραμματικής **S** παράγει μία δεδομένη λέξη της γλώσσας
- Εάν σε κάποιο σημείο της παραγωγής υπάρχει κάποιο μη τερματικό σύμβολο **A** το οποίο αντικαθίσταται βάση ενός κανόνα **$A \rightarrow XYZ$**
 - τότε το δέντρο θα περιλαμβάνει έναν εσωτερικό κόμβο **A** με τρία παιδιά **X**, **Y** και **Z**, από τα αριστερά προς τα δεξιά





Δέντρα συντακτικής ανάλυσης (2/4)

■ Ορισμός

Δεδομένης μίας γραμματικής ανεξάρτητης συμφραζομένων, ένα δέντρο συντακτικής ανάλυσης είναι ένα δέντρο με τις παρακάτω ιδιότητες:

- Η ρίζα του δέντρου ονομάζεται με το αρχικό σύμβολο της γραμματικής.
- Κάθε φύλλο ονομάζεται από ένα τερματικό σύμβολο ή το ε .
- Κάθε εσωτερικός κόμβος ονομάζεται από ένα μη τερματικό σύμβολο.
- Έστω A κάποιος εσωτερικός κόμβος και $X_1, X_2, \dots, X_n$ τα παιδιά του κόμβου αυτού, από αριστερά προς τα δεξιά.
 - Τότε το $A \rightarrow X_1 X_2 \dots X_n$ είναι ένας γραμματικός κανόνας, με X_i να είναι γραμματικό σύμβολο (τερματικό ή μη).
 - Στην ειδική περίπτωση που έχουμε $A \rightarrow \varepsilon$, τότε ο κόμβος A έχει ένα μόνο παιδί ε , το οποίο είναι και φύλλο.

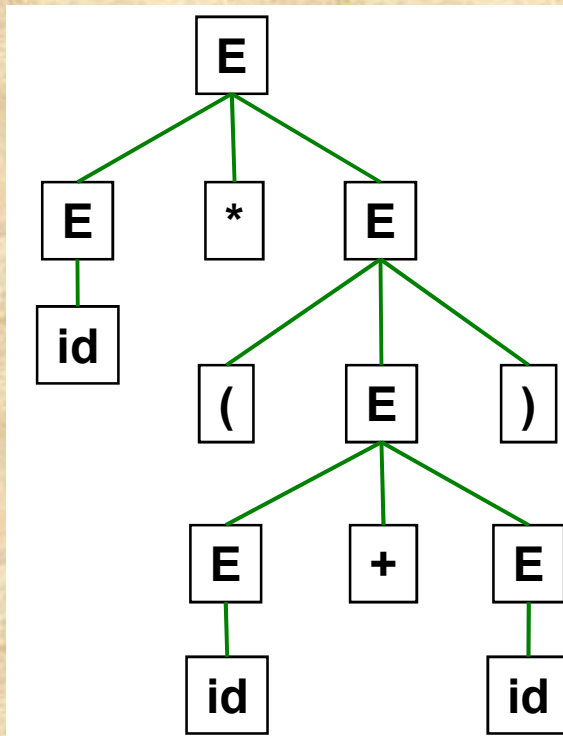
Δέντρα συντακτικής ανάλυσης (3/4)

■ Παραδείγματα (1/2)

Parse trees για το $\text{id}*(\text{id}+\text{id})$ με τις παραγωγές:

$E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (E + E) \Rightarrow \text{id} * (E + E) \Rightarrow \text{id} * (\text{id} + E) \Rightarrow \text{id} * (\text{id} + \text{id})$

$E \Rightarrow E * E \Rightarrow \text{id} * E \Rightarrow \text{id} * (E) \Rightarrow \text{id} * (E + E) \Rightarrow \text{id} * (\text{id} + E) \Rightarrow \text{id} * (\text{id} + \text{id})$

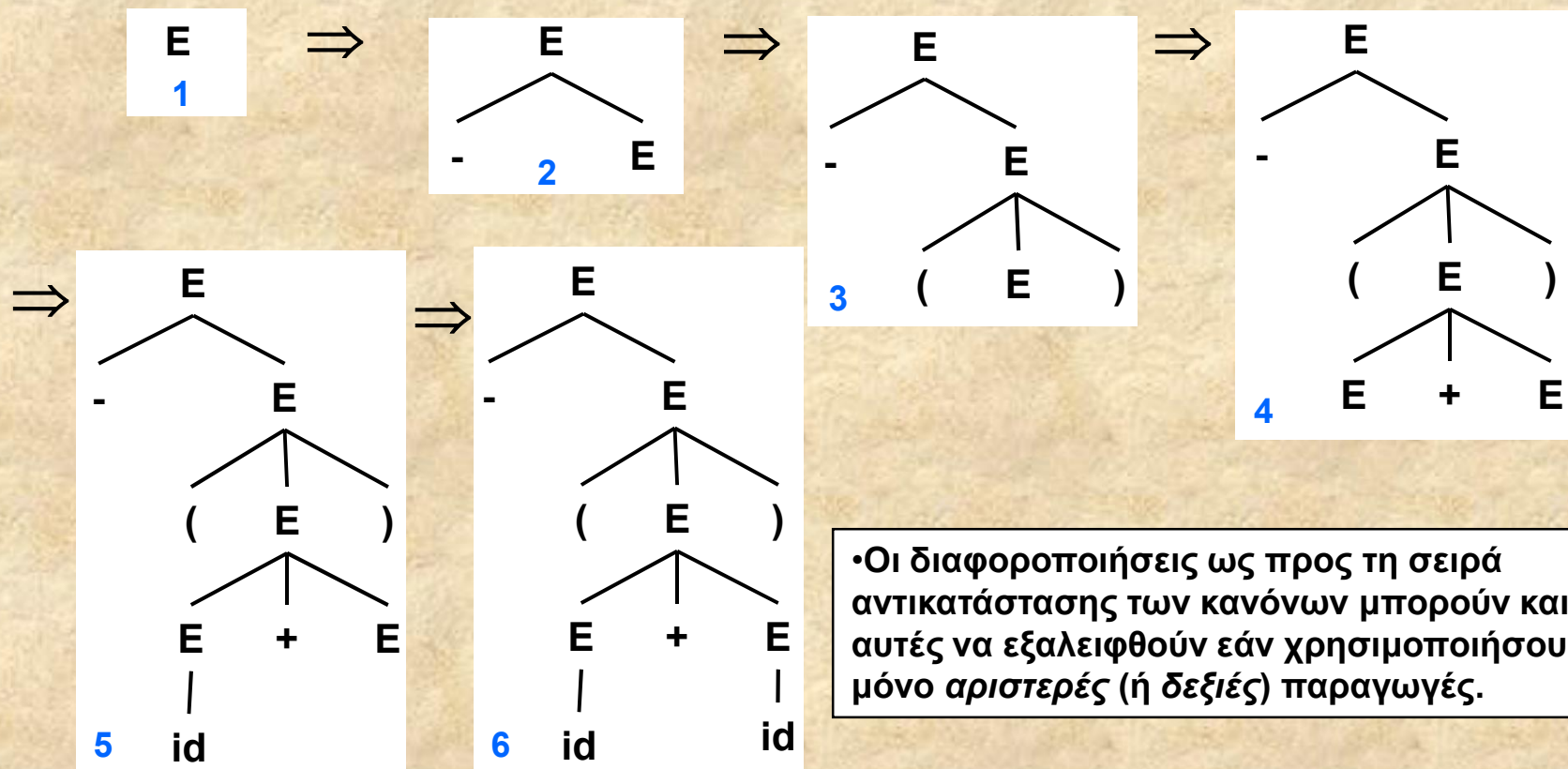


- Αυτό που παρατηρούμε είναι ότι και για τις δύο διαφορετικές παραγωγές καταλήγουμε στο ίδιο ακριβώς parse tree!
- Το παραπάνω αντικατοπτρίζει μία βασική ιδιότητα των parse trees, ότι δηλ. **φιλτράρουν τις επιλογές σχετικά με τη σειρά που εφαρμόζονται οι αντικαταστάσεις χρησιμοποιώντας γραμματικούς κανόνες.**
- Αν θέλουμε μπορούμε να ορίσουμε ειδικά parse trees όπου κάθε εσωτερικός κόμβος φέρει και τη μοναδική θέση του κανόνα αντικατάστασης στη συνολική παραγωγή
- Ωστόσο, όπως θα δούμε, δεν είναι πάντα εξασφαλισμένο ότι για την ίδια πρόταση της γλώσσας θα υπάρχει ένα και μοναδικό parse tree.

Δέντρα συντακτικής ανάλυσης (4/4)

■ Παραδείγματα (2/2)

Διαδοχική κατασκευή του parse trees για το $-(id+id)$ με την παραγωγή:
 $E \Rightarrow -E \Rightarrow -(E) \Rightarrow -(E+E) \Rightarrow -(id+E) \Rightarrow -(id+id)$



•Οι διαφοροποιήσεις ως προς τη σειρά αντικατάστασης των κανόνων μπορούν και αυτές να εξαλειφθούν εάν χρησιμοποιήσουμε μόνο *αριστερές* (ή *δεξιές*) παραγωγές.

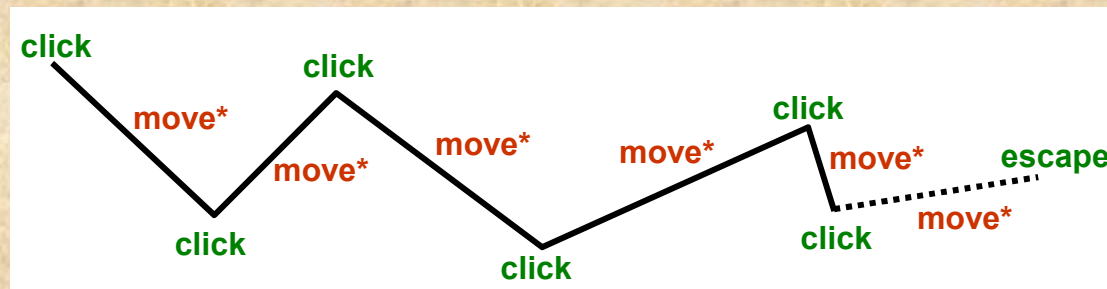


Ένθετο – parse trees (1/2)

Η παρακάτω γραμματική περιγράφει το συντακτικό για τη ζωγραφική ενός πολυγώνου στις γλώσσας εισόδου του συστήματος διεπαφής (User Interface) ενός graphic editor.

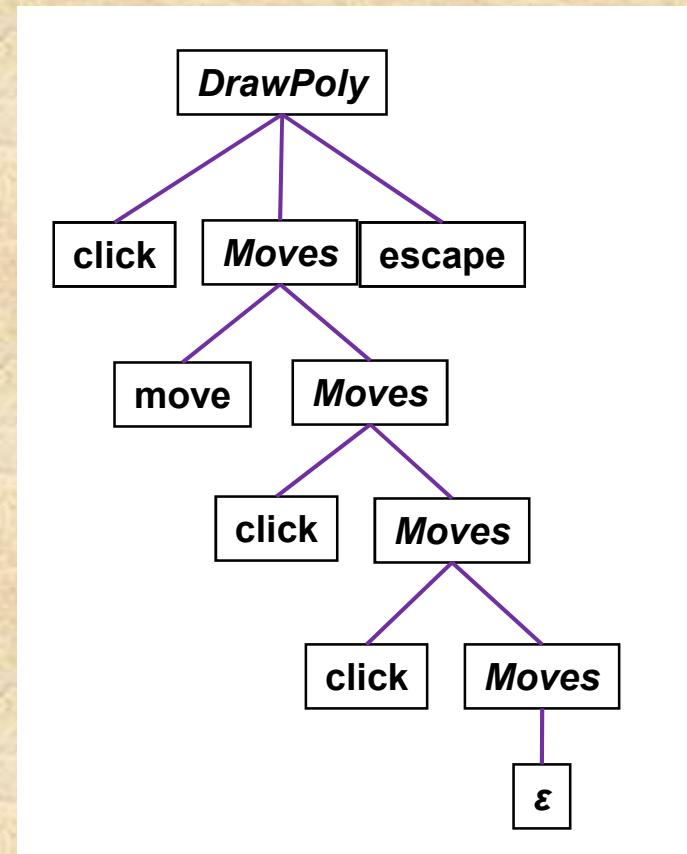
DrawPoly $\rightarrow$ **click** *Moves* **escape**
Moves $\rightarrow$ **click** *Moves* | **move** *Moves* | ϵ

DrawPoly $\Rightarrow lm$ **click** *Moves* **escape** *Με αριστερές παραγωγές μόνο*
 $\Rightarrow lm$ **click** **move** *Moves* **escape**
 $\Rightarrow lm$ **click** **move** **move** *Moves* **escape**
 $\Rightarrow lm$ **click** **move** **move** **click** *Moves* **escape**
 $\Rightarrow lm$ **click** **move** **move** **click** ϵ **escape**



Ένθετο – parse trees (2/2)

- Η είσοδος (tokens ή terminals) προέρχεται από τις συσκευές εισόδου.
- Ο μεταφραστής βασίζεται σε αυστηρά καθορισμένο συντακτικό ενεργειών του χρήστη.
- Οι ενέργειες μετάφρασης αφορούν την μετατροπή των *προτάσεων* της γλώσσας εισόδου στα αντίστοιχα γραφικά στοιχεία.
- Η μετάφραση (δηλ. δημιουργία του πολυγώνου) γίνεται παράλληλα με την είσοδο και την ανάλυση του συντακτικού – με κάθε *click* token.





Περιεχόμενα

- Γραμματικές ανεξάρτητες συμφραζομένων
- Δέντρα συντακτικής ανάλυσης
- *Εξάλειψη διφορούμενης ανάλυσης*



Εξάλειψη διφορούμενης ανάλυσης (1/16)

- Εάν για μία λέξη της γλώσσας δύναται να προσδιοριστούν περισσότερα τους ενός δέντρα συντακτικής ανάλυσης
 - *Τότε η γραμματική αυτή ονομάζεται διφορούμενη*
 - Η ύπαρξη τέτοιου χαρακτηριστικού είναι άκρως ανεπιθύμητη, καθώς κάθε διαφορετικό δέντρο συντακτικής ανάλυσης στην πράξη αντιστοιχεί και σε διαφορετική «ερμηνεία» της λέξης εισόδου
 - ◆ Δηλ. κυριολεκτικά είναι διφορούμενη η γραμματική μας
 - Για την κατασκευή μεταφραστών είναι απαραίτητο να υφίσταται μόνο μία ερμηνεία για οποιαδήποτε λέξη της γλώσσας



Εξάλειψη διφορούμενης ανάλυσης (2/16)

- **Πόρισμα:** Εάν μία γραμματική δεν είναι διφορούμενη, τότε όλες οι παραγωγές της ίδιας δεδομένης λέξης έχουν ακριβώς τον ίδιο αριθμό βημάτων.
- **Απόδειξη:** Για κάθε λέξη έχουμε ένα parse tree. Κάθε υπόδεντρο, δηλ. κόμβος και άμεσοι απόγονοι, αντιπροσωπεύει μία παραγωγή σε ένα βήμα. Στο δέντρο έχουμε N τέτοια υπόδεντρα, που είναι και οι παραγωγές της λέξη αυτής.



Εξάλειψη διαφορούμενης ανάλυσης (3/16)

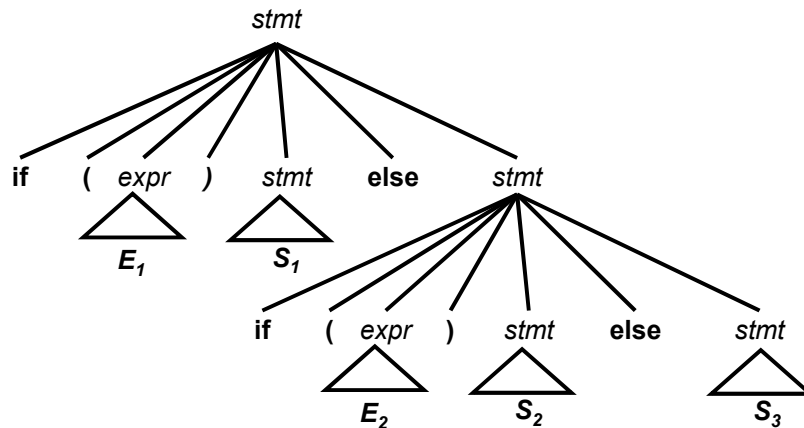
- Ένα κλασικό παράδειγμα διαφορούμενης γραμματικής (1/2)

$stmt \rightarrow$ $if (expr) stmt$
| $if (expr) stmt else stmt$
| $other$

Η γραμματική που περιγράφει εντολές διακλάδωσης με συντακτικό παρόμοιο με αυτό της C

$if (E_1) S_1 else if (E_2) S_2 else S_3$

Ένας προτασιακός τύπος της γραμματικής



- Το αντίστοιχο δέντρο συντακτικής ανάλυσης. Στην περίπτωση αυτή είναι και το μοναδικό.
- Η απόδειξη είναι εύκολη και γενικά εφαρμόζεται συνήθως επαγωγικά ή με άτοπο απαγωγή.



Εξάλειψη διφορούμενης ανάλυσης (5/16)

- Για το προηγούμενο παράδειγμα είναι γνωστό ότι σε όλες τις γλώσσες προγραμματισμού που υποστηρίζουν τέτοιου είδους εντολές η πρώτη ερμηνεία προτιμάται.
 - Ο γενικός κανόνας είναι «**αντιστοίχισε κάθε *else* με το πλησιέστερο μη αντιστοιχισμένο *if***»
 - Θα δούμε ότι αυτός ο κανόνας μπορεί να ενσωματωθεί στη γραμματική, μεταβάλλοντας προφανώς τους αρχικούς κανόνες
 - Ωστόσο ο τρόπος με τον οποίο καταλήγουμε να αλλάξουμε τη γραμματική δεν είναι τόσο προφανής και δεν ακολουθεί κάποια τυποποιημένα βήματα
 - ◆ Για το λόγο αυτό τέτοιου είδους μετατροπές είναι γνωστές ως *grammar hacking*
 - Θα δούμε δύο παραδείγματα για *grammar hacking*, ξεκινώντας να λύσουμε πρώτα το πρόβλημα της διφορούμενης *if...else* γραμματικής

Ένθετο – εξάλειψη διαφορούμενης ανάλυσης

if *good_stmt* *else* *good_stmt*

good_stmt

Εάν έχω *if-else* εντολή όπου τα *stmts* είναι *good* δηλ. δεν κλέβουν άλλο *else* όπου και να τοποθετηθούν, τότε το ίδιο μπορεί να θεωρηθεί ως *good*

if

good_stmt *bad_stmt*

bad_stmt

Εάν έχω *if* εντολή τότε το *stmt* μπορεί μεν να είναι *good* ή *bad*, αλλά το ίδιο *if* από μόνο του είναι πρόβλημά καθώς μπορεί να κλέψει κάποιο επόμενο *else*

if *else*

good_stmt *bad_stmt*

bad_stmt

Εάν έχω *if-else* με το *if-stmt* να είναι *good* και το *else-stmt* να είναι *bad*, τότε το *if-else* μπορεί να κλέψει ένα άλλο *else* και είναι *bad*

Για να μην «κλέψει» το *if* κάποιο επόμενο *else* βάλουμε πάντα μετά το *if* ήδη ζευγαρωμένα *else* (τα *good stmts*)



Εξάλειψη διφορούμενης ανάλυσης (6/16)

Hint: για να επιβάλλω τον κανόνα “else με το πλησιέστερο if”, πρέπει να εξασφαλίσω ότι στη νέα γραμματική αυτά τα δύο βρίσκονται πάντα μαζί «πακέτο» στην ίδια παραγωγή, δηλ. στο ίδιο υπό-δέντρο *if (e) s else s*

- Έστω η απλή μορφή εντολής διακλάδωσης **if (E) A else B**, με *A* και *B* εντολές οποιουδήποτε τύπου, δηλ. **other**, εκτός από εντολή διακλάδωσης. Τότε μία τέτοια παραγωγή εξασφαλίζει αντιστοίχιση του **else** με το **if**. Την ονομάζω τύπου *matched_ifelse*.
- Εάν τώρα επιπλέον τα *A* και *B* είναι *matched_ifelse* τότε και πάλι είναι εξασφαλισμένο ότι το **else** αντιστοιχείται πάντα στο αρχικό **if** του κανόνα, καθώς η *matched_ifelse* ποτέ δεν μπορεί τελειώνει σε σκέτο **if**.

matched_ifelse → **if (E) matched_ifelse else matched_ifelse**
| **other**

1

Δεν μπορεί ποτέ σε ένα **if if** το 1^ο να αρπάξει το **else** του 2^{ου}

```
if (b) x-=y; else y+=x;
if (b1) if (b2) z=w; else w=z; else x=z+w;
if (a) if (b) printf("b"); else printf("c");
```

Δεν καλύπτεται
από το νέο
κανόνα

Εκτός ύλης



Εξάλειψη διφορούμενης ανάλυσης (7/16)

- Τώρα ξεχωρίζω την περίπτωση του σκέτου **if** το οποίο δεν ακολουθείται από κάποιο **else** με το οποίο να αντιστοιχείται, δηλ **if (E) stmt**. Μία τέτοια εντολή την ονομάζω όπως αναμένεται *unmatched_if*.

unmatched_if $\rightarrow$ **if (E) stmt**

- Πρέπει να εξασφαλίσουμε ότι αυτό το παραπάνω **if** δεν «κλέβει» το **else** κάποιας εντολής διακλάδωσης φωλιασμένης μέσα στο *stmt*. Για επιτευχθεί αυτό αρκεί *stmt* $\rightarrow$ *matched_ifelse*. Αλλιώς, μπορεί προφανώς το *stmt* να είναι εντολή χωρίς **else**, δηλ. και *stmt* $\rightarrow$ *unmatched_if*

stmt $\rightarrow$ *matched_ifelse* | *unmatched_if*

2

```
if (b) x==y;  
if (a) if (b) z=w; else w = z;  
if (a) if (b) printf("b");  
if (a) z=w; else w=z;
```

Δεν καλύπτεται από το
νέο κανόνα, αλλά είναι
matched_ifelse

Εκτός ύλης



Εξάλειψη διφορούμενης ανάλυσης (8/16)

- Τώρα μένει η περίπτωση στην οποία έχω μία εντολή διακλάδωσης για την οποία μετά το **else** δεν έπεται *matched_ifelse*. Προφανώς μετά από ένα τέτοιο **else** μπορεί να έπεται μόνο μία εντολή *unmatched_if*. Δηλ. πρόκειται για μια εντολή διακλάδωσης του τύπου **if (E) matched_ifelse else unmatched_if**.
- Όμως όπως φαίνεται και μία τέτοια εντολή πάντα θα τελειώνει με ένα **if** το οποίο δεν αντιστοιχείται, άρα είναι και αυτή η εντολή της κατηγορίας *unmatched_if*, έτσι έχω:

unmatched_if → **if (E) stmt | if (E) matched_ifelse else unmatched_if**

3

```
if (b) x-=y; else if (a) y=z;  
if (a) if (b) z=w; else w = z; else if (c) x=0;  
if (a) if (b) printf("b"); else printf("other");
```

Δεν καλύπτεται
από το νέο
κανόνα.



Εξάλειψη διφορούμενης ανάλυσης (9/16)

- Οι διφορούμενες γραμματικές δεν είναι δύσκολο να προκύψουν, καθώς δεν υπάρχει κάποιο τυποποιημένο «κατασκευαστικό συνταγολόγιο» ώστε να αποφεύγουμε τη γένεση τους
 - Ακόμη και η διαπίστωση (απόδειξη) του γεγονότος ότι μία συγκεκριμένη γραμματική είναι διφορούμενη είναι δύσκολη
 - Αφού απαιτεί τον εντοπισμό κάποιας συγκεκριμένης λέξης που παράγει πλέον του ενός δέντρα συντακτικής ανάλυσης
 - ◆ Ο προσδιορισμός μίας τέτοιας λέξης (αντιπαράδειγμα) δεν είναι τυποποιημένος
 - Θα δούμε μία ακόμη διφορούμενη γραμματική για απλές αριθμητικές εκφράσεις, μελετώντας λεπτομερώς την τακτική μετατροπής των αρχικών γραμματικών κανόνων, μέσω grammar hacking, ώστε να εξαλείψουμε τα διφορούμενα χαρακτηριστικά



Εξάλειψη διφορούμενης ανάλυσης (10/16)

Η γραμματική και οι κανονικές
εκφράσεις για τα tokens

$S \rightarrow expr$
 $expr \rightarrow expr \text{ op } expr$
 $expr \rightarrow \text{intnum}$
 $expr \rightarrow (expr)$

$\text{op} = + \mid - \mid * \mid /$
 $\text{intnum} = \text{digit}^+$

Μία αριστερή παραγωγή της
λέξης $(2-1)+1$

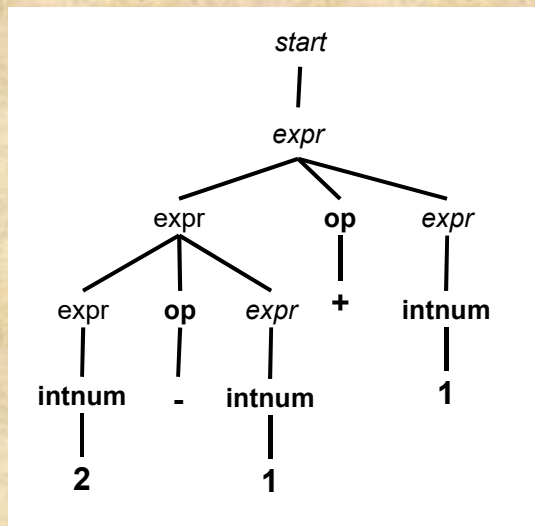
S	$\Rightarrow expr$	$\Rightarrow$
$expr \text{ op } expr$	$\Rightarrow (expr) \text{ op } expr$	$\Rightarrow$
$(expr \text{ op } expr) \text{ op } expr$	$\Rightarrow (\text{intnum op } expr) \text{ op } expr$	$\Rightarrow$
$(2 \text{ op } expr) \text{ op } expr$	$\Rightarrow (2 - expr) \text{ op } expr$	$\Rightarrow$
$(2 - \text{intnum}) \text{ op } expr$	$\Rightarrow (2 - 1) \text{ op } expr$	$\Rightarrow$
$(2 - 1) + expr$	$\Rightarrow (2 - 1) + \text{intnum}$	$\Rightarrow$
$(2 - 1) + 1$		



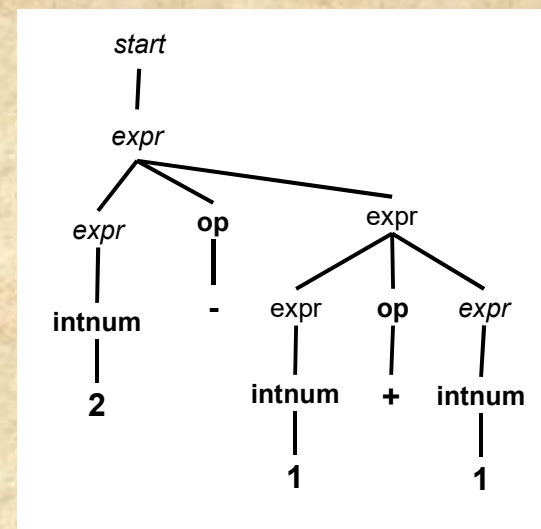
Εξάλειψη διφορούμενης ανάλυσης (11/16)

- Υπάρχουν δύο διαφορετικά εφικτά δέντρα συντακτικής ανάλυσης για την έκφραση **2-1+1**, το ένα να αντιστοιχεί στην ερμηνεία **(2-1)+1**, το δε άλλο στην ερμηνεία **2-(1+1)**

Δέντρο που αντιστοιχεί στην
ερμηνεία (2-1)+1



Δέντρο που αντιστοιχεί στην
ερμηνεία 2-(1+1)



Εδώ φαίνεται ότι για να λυθεί το πρόβλημα πρέπει η απλή γραμματική μας να υιοθετήσει μόνο ένα είδος προσηταιριστικότητας για τους τελεστές (δηλ. αριστερά ή δεξιά)



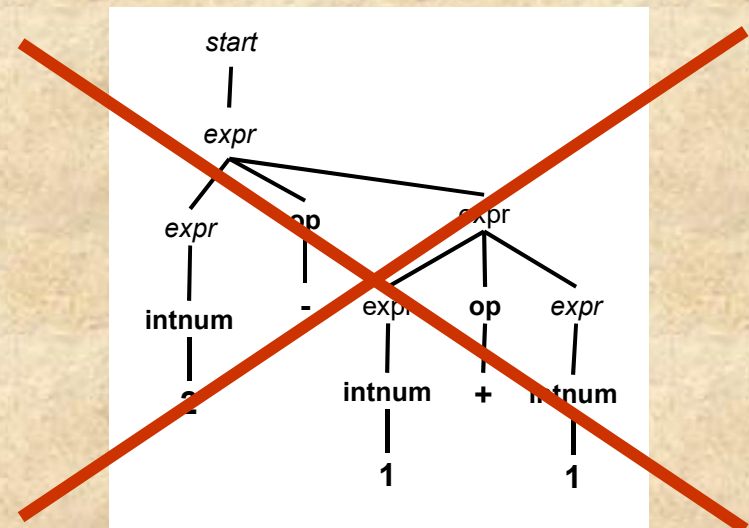
Εξάλειψη διφορούμενης ανάλυσης (12/16)

- Η αλλαγή που κάνουμε στη γραμματική είναι να επιβάλλουμε κανόνα αριστερής προσηταιριστικότητας
 - Αυτό σημαίνει ότι η παραγωγή θα μπορεί να εφαρμόζεται επαναληπτικά μόνο στο αριστερό *expr*, άρα το δεξί πρέπει να γίνει **intnum**

$S \rightarrow expr$
 $expr \rightarrow expr \text{ op intnum}$
 $expr \rightarrow \text{intnum}$
 $expr \rightarrow (expr)$

 $op = + \mid - \mid * \mid /$
 $\text{intnum} = \text{digit}^+$

Τώρα το παρακάτω δέντρο δεν μπορεί να προκύψει από την τροποποιημένη γραμματική για τη λέξη 2-1+1

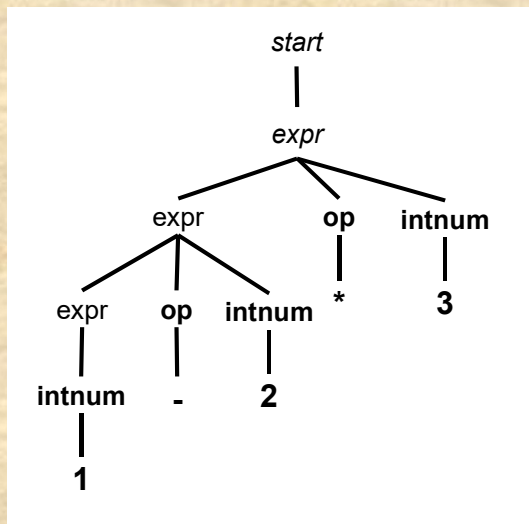




Εξάλειψη διαφορούμενης ανάλυσης (13/16)

- Η νέα αυτή γραμματική αν και δεν είναι διαφορούμενη είναι σημασιολογικά λανθασμένη καθώς παραβαίνει τους γνωστούς κανόνες προτεραιότητας των τελεστών
 - Π.χ. η έκφραση $1-2*3$ παράγει δέντρο συντακτικής ανάλυσης που αντιστοιχεί στην ερμηνεία $(1-2)*3$ και όχι στη σωστή $1-(2*3)$

Το δέντρο ανάλυσης για τη
λέξη $1-2*3$ με ερμηνεία $(1-2)*3$





Εξάλειψη διφορούμενης ανάλυσης (14/16)

- Για να μετατρέψουμε αυτή τη γραμματική ώστε να σέβεται τους κανόνες προτεραιότητας εφαρμόζουμε την εξής τακτική:

Ομαδοποιούμε τους δυαδικούς τελεστές σε διαδοχικά αριθμημένα επίπεδα προτεραιότητας, από το υψηλότερο προς το χαμηλότερο, ως εξής:

- Οι αρχικές μοναδιαίες εκφράσεις στο 0° επίπεδο
- Το $*$ και το $/$ στο 1° επίπεδο
- Το $+$ και το $-$ στο 2° επίπεδο

↓
Μείωση
προτεραιότητας

Ορίζουμε νέα μη τερματικά σύμβολα E_j για τις εκφράσεις, ένα ανά επίπεδο j , δηλ. στην προκειμένη περίπτωση τρία νέα τέτοια σύμβολα E_0 , E_1 και E_2 . Γενικά ισχύει:

- $\forall i, j$ αριθμούς επιπέδων και **op** τελεστή επιπέδου j ώστε να είναι $j = i+1$, ορίζουμε τους κανόνες:
 - $E_j \rightarrow E_j \text{ op } E_i$ (για αριστερή προσεταιριστικότητα)
 - $E_j \rightarrow E_i$



Εξάλειψη διφορούμενης ανάλυσης (15/16)

Δηλ. το γραμματικό σύμβολο που χρησιμοποιείται για τελεστές σε χαμηλότερο επίπεδο προτεραιότητας μπορεί μόνο να παράγει το γραμματικό σύμβολο των τελεστών με την αμέσως μεγαλύτερη προτεραιότητα.

- Αυτό σημαίνει στην πράξη ότι οι δυαδικοί τελεστές με την μεγαλύτερη προτεραιότητα εμφανίζονται πάντα χαμηλότερα στο δέντρο, πλησιέστερα είτε στα τερματικά σύμβολα ή σε μοναδιαίες εκφράσεις μεγαλύτερης προτεραιότητας (π.χ. παρενθέσεις ή μοναδιαίο -).
- Ενώ το αρχικό σύμβολο παράγει το τερματικό σύμβολο χαμηλότερης προτεραιότητας.

*Υψηλή
προτεραιότητα =
κλέβω τα tokens
πριν τους άλλους*

*Υψηλή
προτεραιότητα =
βρίσκομαι στο
δέντρο πιο κοντά
στα φύλλα*

S	→ <i>expr2</i>	
<i>expr2</i>	→ <i>expr2 addsub expr1</i>	Επίπεδο +, - αριστερή προσεταιριστικότητα
<i>expr2</i>	→ <i>expr1</i>	
<i>expr1</i>	→ <i>expr1 muldiv expr0</i>	Επίπεδο *, / αριστερή προσεταιριστικότητα
<i>expr1</i>	→ <i>expr0</i>	
<i>expr0</i>	→ (<i>expr2</i>)	Επίπεδο «μοναδιαίων» εκφράσεων
<i>expr0</i>	→ intnum	



Εξάλειψη διφορούμενης ανάλυσης (16/16)

Το δέντρο ανάλυσης της τροποποιημένης γραμματικής για τη λέξη 1-2*3 έχει τώρα τη σωστή ερμηνεία 1-(2*3)

